

Harness Engineering in Scrutinize

AI Architecture Team

Document Version: 1.0

Status: Active / Architectural Standard

1 What is “Harness Engineering” and Why Is It Called a Harness?

1.1 Why Call It a “Harness”?

In physical engineering, a **harness** (such as a safety harness, wiring harness, or horse harness) is a structural framework that constrains, directs, secures, and channels raw power into predictable, controlled work.

In modern generative AI and agentic software (particularly across Scrutinize V4 and V5):

Harness Engineering is the design pattern of building a strict, deterministic, policy-controlled, and observable safety & validation framework *around* non-deterministic Large Language Models (LLMs) and tools.

Instead of letting an LLM run directly as a loose, unpredictable chatbot (where bad outputs, hallucinated citations, prompt injections, or infinite loops can break the system), the LLM operates inside an **engineered software harness**.

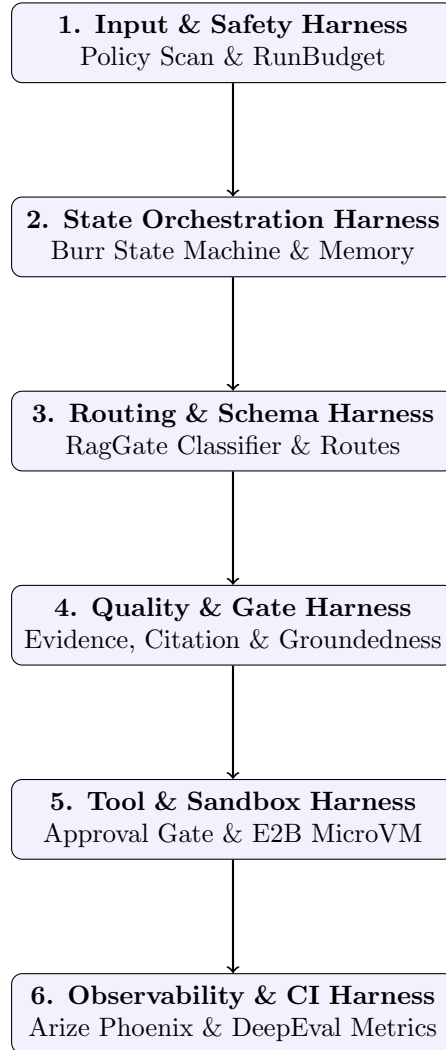
The harness acts as a **containment vessel**:

1. **Constrains inputs:** Scans and sanitizes user queries and retrieved context before the LLM sees them.
 2. **Structures orchestration:** Enforces explicit state machine transitions (Apache Burr) instead of unmonitored recursive agent calls.
 3. **Validates outputs:** Subjects model drafts to automated evidence, citation, and groundedness gates before sending data to the client.
 4. **Controls execution:** Enforces strict run budgets, role-based tool permission policies, and human-in-the-loop approval gates.
 5. **Isolates risk:** Executes untrusted code inside Firecracker MicroVM sandboxes (E2B).
-

2 Comprehensive Trace of Harness Engineering Across the System

Harness engineering principles are embedded deeply across Scrutinize’s backend, middleware, database, and evaluation layers. Below is a structured trace of how harness engineering is realized in the architecture.

2.1 Architecture Execution Flow



2.2 Component Trace Details

2.2.1 2.1 State Orchestration & State Machine Harness

- **Harness Mechanism:** Rather than writing arbitrary async LLM loops, the entire query lifecycle is compiled into an **Apache Burr** state machine.
- **Why it matters:** Burr provides explicit state transitions (`input_validated` → `routed` → `retrieved` → `validated_citations` → `responded`). State is immutable, step execution is deterministic, and execution can be paused for user interaction.

2.2.2 2.2 Input Safety & Injection Containment Harness

- **Harness Mechanism:** Validates incoming text and tool call actions against security rules prior to pipeline execution.
- **Why it matters:** Retrieved web content or user prompts can contain prompt injection attacks (“ignore previous instructions”). The input harness acts as a security firewall.

2.2.3 2.3 Type Safety & Schema Harness

- **Harness Mechanism:** Uses **PydanticAI** and strict Pydantic schemas for classification and agent output parsing.
- **Why it matters:** LLMs natively output loose string tokens. By wrapping prompts with Pydantic schemas, the harness enforces strict JSON output contracts. If the LLM generates invalid fields, PydanticAI automatically catches the validation error and prompts the LLM to auto-correct.

2.2.4 2.4 Deterministic Quality Gates (Evidence, Citations, Groundedness)

The core of Scrutinize’s RAG trust model relies on three sequential downstream quality gates:

1. Evidence Sufficiency Gate

- **Mechanism:** Evaluates retrieved vector search chunks before synthesis. If retrieved facts do not actually contain the answer, the harness forces an immediate **abstention flow** instead of allowing the LLM to hallucinate.

2. Citation Verifier Gate

- **Mechanism:** Parses claims and claim source IDs [`doc_id`]. Deterministically validates whether the cited chunk exists and semantically supports the claim. If invalid, it triggers targeted regeneration.

3. Groundedness Evaluator Gate

- **Mechanism:** Computes a numerical groundedness score (0.0 to 1.0). If the score falls below threshold (≥ 0.90), the harness rejects the response draft and loops back to synthesis.

2.2.5 2.5 Resource & Budget Limitation Harness

- **Harness Mechanism:** Tracks max iterations, maximum token budgets, cost limits, and wall-clock execution time per run.
- **Why it matters:** Prevents runaway agent loops, accidental high API bills, or infinite retry loops during failure states.

2.2.6 2.6 Sandboxed Execution & Approval Harness

- **Harness Mechanism:** Sensitive tool invocations require approval entries in PostgreSQL and SSE notification to the client. Executable code generated by LLMs runs exclusively inside **E2B Sandboxes** (isolated Firecracker MicroVMs).
- **Why it matters:** Ensures that generated Python code or web execution cannot access system memory, environment variables, or private networks on the host backend server.

2.2.7 2.7 Temporal Memory & Context Boundary Harness

- **Harness Mechanism:** Integrates **Letta** (Core and Archival Memory) and **Graphiti** (Temporal Knowledge Graphs).
- **Why it matters:** Instead of dumping unformatted chat histories into context windows (which degrades model performance and introduces stale facts), the memory harness structures evolving project knowledge deterministically across temporal state graphs.

2.2.8 2.8 Evaluation & Benchmark Harness

- **Harness Mechanism:** Defines standardized measurement pipelines for NDCG@k, MRR, Hit Rate, precision, recall, and adversarial security testing.
 - **Why it matters:** Allows developers to run CI/CD regression tests against retrieval quality and system prompts before deploying updates to production.
-

3 Summary Matrix of Harness Controls

Harness Layer	Primary Purpose / Control
State Machine Harness	Explicit step-by-step state machine execution via Burr
Type & Schema Harness	PydanticAI type enforcement and self-correcting outputs
Evidence Gate Harness	Pre-synthesis retrieval relevance check & forced abstention
Citation Verification Harness	Factual claim-level verification against vector sources
Groundedness Harness	Faithfulness scoring and rejection threshold (≥ 0.90)
Budget Control Harness	Token count, cost limit, iteration count, and timeout limits
Security & Sandbox Harness	E2B MicroVM execution & sensitive action approvals
Memory State Harness	Letta permanent memory state and Graphiti temporal fact graphs
Evaluation Harness	Automated metrics (NDCG@k, Hit Rate, MRR) and DeepEval CI

Table 1: Summary of Harness Engineering Controls across Scrutinize.

4 Architectural Conclusion

In Scrutinize, **Harness Engineering** is not just an auxiliary testing framework—it is the **primary runtime control architecture**. It transforms probabilistic LLM generations into deterministic enterprise software by placing safety, verification, memory, budget, and execution harnesses around every generative call.